

Comprehensive Graph Algorithms Guide 6

Table of Contents

1. BFS Applications
 2. Shortest Path Algorithms
 3. Advanced Graph Problems
 4. Algorithm Comparison
 5. Implementation Tips
-

BFS Applications

1. Minimum Multiplications to Reach Target

Problem Statement: Given a start number, end number, and an array `arr`, find the minimum number of multiplications needed to reach the end by repeatedly multiplying with any number in `arr`, taking modulo 100000 at each step.

Key Insights:

- This is essentially finding shortest path in an unweighted graph
- Each number from 0 to 99999 represents a node
- Edges exist between numbers that can be reached by one multiplication
- BFS guarantees minimum steps due to unweighted nature

Implementation:

```
int minimumMultiplications(vector<int>& arr, int start, int end) {  
    const int MOD = 100000;  
  
    if (start == end) return 0;  
  
    queue<pair<int, int>> q; // {current_number, steps}  
    vector<bool> visited(MOD, false);  
  
    q.push({start, 0});  
    visited[start] = true;  
  
    while (!q.empty()) {  
        auto [node, steps] = q.front();  
        q.pop();  
        if (node == end) return steps;  
        for (int num : arr) {  
            int next = (node * num) % MOD;  
            if (!visited[next]) {  
                q.push({next, steps + 1});  
                visited[next] = true;  
            }  
        }  
    }  
    return -1;  
}
```

```

        auto [node, steps] = q.front();
        q.pop();

        for (int multiplier : arr) {
            int newNode = (node * multiplier) % MOD;

            if (newNode == end) {
                return steps + 1;
            }

            if (!visited[newNode]) {
                visited[newNode] = true;
                q.push({newNode, steps + 1});
            }
        }
    }

    return -1; // Target unreachable
}

```

Time Complexity: $O(\text{MOD} + \text{arr.size()} \times \text{MOD}) = O(100000 \times \text{arr.size()})$ **Space Complexity:** $O(\text{MOD}) = O(100000)$



Shortest Path Algorithms

2. Dijkstra's Algorithm with Path Counting

Problem: Count Paths from Node 0 to n-1 in Shortest Time (Leetcode 1976)

Approach:

- Modified Dijkstra's algorithm
- Maintain both distance and path count arrays
- When finding equal distance paths, add to the count

Implementation:

```

int countPaths(int n, vector<vector<int>>& roads) {
    const int MOD = 1e9 + 7;

    // Build adjacency list
    vector<vector<pair<int, long long>>> adj(n);
    for (auto& road : roads) {
        adj[road[0]].push_back({road[1], road[2]});
        adj[road[1]].push_back({road[0], road[2]});
    }

    vector<long long> dist(n, LLONG_MAX);

```

```

vector<long long> ways(n, 0);

dist[0] = 0;
ways[0] = 1;

priority_queue<pair<long long, int>,
               vector<pair<long long, int>>,
               greater<>> pq;
pq.emplace(0, 0);

while (!pq.empty()) {
    auto [currDist, node] = pq.top();
    pq.pop();

    if (currDist > dist[node]) continue;

    for (auto& [neighbor, cost] : adj[node]) {
        long long newDist = currDist + cost;

        if (newDist < dist[neighbor]) {
            dist[neighbor] = newDist;
            ways[neighbor] = ways[node];
            pq.emplace(newDist, neighbor);
        } else if (newDist == dist[neighbor]) {
            ways[neighbor] = (ways[neighbor] + ways[node]) % MOD;
        }
    }
}

return ways[n-1];
}

```

3. Bellman-Ford Algorithm

Purpose:

- Single-source shortest path with negative edge weights
- Detects negative weight cycles
- Works on directed graphs

Key Concepts:

- Relaxes all edges V-1 times (V = number of vertices)
- If distances can still be improved in the V-th iteration, negative cycle exists

Implementation:

```

vector<int> bellmanFord(int V, vector<vector<int>>& edges, int src) {
    vector<int> dist(V, 1e8);
    dist[src] = 0;

```

```

// Relax all edges V-1 times
for (int i = 1; i <= V - 1; i++) {
    for (auto& edge : edges) {
        int u = edge[0], v = edge[1], wt = edge[2];
        if (dist[u] != 1e8 && dist[u] + wt < dist[v]) {
            dist[v] = dist[u] + wt;
        }
    }
}

// Check for negative cycles
for (auto& edge : edges) {
    int u = edge[0], v = edge[1], wt = edge[2];
    if (dist[u] != 1e8 && dist[u] + wt < dist[v]) {
        return {-1}; // Negative cycle detected
    }
}

return dist;
}

```

Time Complexity: $O(V \times E)$ **Space Complexity:** $O(V)$

4. Floyd-Warshall Algorithm

Purpose:

- All-pairs shortest paths
- Handles negative weights (but not negative cycles)
- Dynamic programming approach

Key Insight: For each pair of vertices (i,j), consider if going through vertex k gives a shorter path.

Implementation:

```

void floydWarshall(vector<vector<int>>& dist, int n) {
    // dist[i][j] initially contains direct edge weights
    // or infinity if no direct edge exists

    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] == 1e9 || dist[k][j] == 1e9) continue;
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
            }
        }
    }
}

```

Time Complexity: $O(V^3)$ Space Complexity: $O(V^2)$

Cycle Detection

5. Negative Weight Cycle Detection

Using Bellman-Ford:

```
bool hasNegativeCycle(int V, vector<vector<int>>& edges) {
    vector<int> dist(V, 1e8);
    dist[0] = 0; // Assuming source is node 0

    // Relax edges V-1 times
    for (int i = 1; i <= V - 1; i++) {
        for (auto& edge : edges) {
            int u = edge[0], v = edge[1], wt = edge[2];
            if (dist[u] != 1e8 && dist[u] + wt < dist[v]) {
                dist[v] = dist[u] + wt;
            }
        }
    }

    // Check if any edge can still be relaxed
    for (auto& edge : edges) {
        int u = edge[0], v = edge[1], wt = edge[2];
        if (dist[u] != 1e8 && dist[u] + wt < dist[v]) {
            return true; // Negative cycle exists
        }
    }

    return false;
}
```

6. Finding Nodes in Negative Cycle

Advanced Implementation:

```
vector<int> findNegativeCycle(int V, vector<vector<int>>& edges) {
    vector<int> dist(V, 1e8);
    vector<int> parent(V, -1);

    dist[0] = 0;

    int lastUpdated = -1;

    // Run Bellman-Ford and track last updated node
```

```

for (int i = 0; i < V; i++) {
    lastUpdated = -1;
    for (auto& edge : edges) {
        int u = edge[0], v = edge[1], wt = edge[2];
        if (dist[u] != 1e8 && dist[u] + wt < dist[v]) {
            dist[v] = dist[u] + wt;
            parent[v] = u;
            lastUpdated = v;
        }
    }
}

if (lastUpdated == -1) {
    return {}; // No negative cycle
}

// Move to a node definitely in the cycle
for (int i = 0; i < V; i++) {
    lastUpdated = parent[lastUpdated];
}

// Extract the cycle
vector<int> cycle;
for (int v = lastUpdated; ; v = parent[v]) {
    cycle.push_back(v);
    if (v == lastUpdated && cycle.size() > 1) break;
}

reverse(cycle.begin(), cycle.end());
return cycle;
}

```



Advanced Graph Problems

7. Find the City with Fewest Reachable Neighbors (Leetcode 1334)

Problem: Given n cities and a distance threshold, find the city with the fewest number of cities reachable within the threshold. If there's a tie, return the city with the greatest index.

Floyd-Warshall Approach:

```

int findTheCity(int n, vector<vector<int>>& edges, int distanceThreshold) {
    // Initialize distance matrix
    vector<vector<int>> dist(n, vector<int>(n, INT_MAX));

    // Distance from city to itself is 0

```

```

    for (int i = 0; i < n; i++) {
        dist[i][i] = 0;
    }

    // Fill direct edges
    for (auto& edge : edges) {
        dist[edge[0]][edge[1]] = edge[2];
        dist[edge[1]][edge[0]] = edge[2];
    }

    // Floyd-Warshall
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] != INT_MAX && dist[k][j] != INT_MAX) {
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
                }
            }
        }
    }

    // Find city with minimum reachable cities
    int minCity = -1, minCount = INT_MAX;

    for (int i = 0; i < n; i++) {
        int count = 0;
        for (int j = 0; j < n; j++) {
            if (i != j && dist[i][j] <= distanceThreshold) {
                count++;
            }
        }

        if (count <= minCount) {
            minCount = count;
            minCity = i;
        }
    }

    return minCity;
}

```

Alternative Dijkstra Approach:

```

int findTheCityDijkstra(int n, vector<vector<int>>& edges, int distanceThreshold) {
    // Build adjacency list
    vector<vector<pair<int, int>>> adj(n);
    for (auto& edge : edges) {
        adj[edge[0]].push_back({edge[1], edge[2]});
        adj[edge[1]].push_back({edge[0], edge[2]});
    }
}

```

```

auto dijkstra = [&](int src) -> int {
    vector<int> dist(n, INT_MAX);
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq;

    dist[src] = 0;
    pq.push({0, src});

    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();

        if (d > dist[u]) continue;

        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }

    int count = 0;
    for (int i = 0; i < n; i++) {
        if (i != src && dist[i] <= distanceThreshold) {
            count++;
        }
    }
    return count;
};

int minCity = -1, minCount = INT_MAX;
for (int i = 0; i < n; i++) {
    int count = dijkstra(i);
    if (count <= minCount) {
        minCount = count;
        minCity = i;
    }
}

return minCity;
}

```

Algorithm Comparison

Feature	Dijkstra	Bellman-Ford	Floyd-Warshall

Time Complexity	$O((V + E) \log V)$	$O(V \times E)$	$O(V^3)$
Space Complexity	$O(V + E)$	$O(V)$	$O(V^2)$
Negative Weights	❌ No	✅ Yes	✅ Yes
Negative Cycles	❌ Cannot detect	✅ Can detect	❌ Cannot handle
Graph Type	Weighted, non-negative	Any weighted	Any weighted (no negative cycles)
Output	Single-source shortest paths	Single-source shortest paths	All-pairs shortest paths
Implementation	Priority Queue (Min-Heap)	Edge relaxation loop	Triple nested loop
Best Use Case	GPS navigation, network routing	Currency arbitrage, finding negative cycles	Small graphs, preprocessing



Implementation Tips

Graph Representation

Adjacency List (Recommended for most cases):

```
// For weighted graphs
vector<vector<pair<int, int>>> adj(n); // adj[u] = {neighbor, weight}

// For unweighted graphs
vector<vector<int>> adj(n);

// Adding edges
adj[u].push_back({v, weight}); // Directed
adj[v].push_back({u, weight}); // Undirected (add both directions)
```

Edge List (Good for Bellman-Ford):

```
vector<vector<int>> edges; // Each edge: {u, v, weight}
edges.push_back({u, v, weight});
```

Common Patterns

BFS Template:

```
queue<int> q;
vector<bool> visited(n, false);
q.push(start);
visited[start] = true;

while (!q.empty()) {
    int node = q.front();
    q.pop();

    for (int neighbor : adj[node]) {
        if (!visited[neighbor]) {
            visited[neighbor] = true;
            q.push(neighbor);
        }
    }
}
```

Dijkstra Template:

```
vector<int> dist(n, INT_MAX);
priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq;

dist[src] = 0;
pq.push({0, src});

while (!pq.empty()) {
    auto [d, u] = pq.top();
    pq.pop();

    if (d > dist[u]) continue;

    for (auto [v, w] : adj[u]) {
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            pq.push({dist[v], v});
        }
    }
}
```

Memory and Performance Tips

1. Use `vector<bool>` for visited arrays when memory is tight
2. Reserve space for vectors when size is known: `adj.reserve(n)`
3. Use appropriate data types: `long long` for large distances

3. **Use appropriate data types:** `long long` for large distances
4. **Early termination:** Stop when target is reached in single-target problems
5. **Bidirectional search:** For single source-target pairs, can reduce time complexity